

ПРОГРАМНИЙ ПРОЄКТ №2

Тема: дослідження швидкодії алгоритму `std::all_of` та створення його паралельної аналогової реалізації

Виконала: студентка групи К-25, Ткаченко Анастасія

Репозиторій: https://github.com/sshmdu/lab2_var1_k25.git

Мета: дослідити `std::all_of` без політики та з політиками `seq`, `par`, `unseq`, `par_unseq`, а також власну реалізацію на `std::jthread` з розбиттям масиву на K блоків. Дані: цілі числа $1 \dots 1000$ (`mt19937`, `seed 12345`), $N = 10^5$, $5 \cdot 10^5$, 10^6 , $5 \cdot 10^6$, збережені у `data_N.txt` і однакові для всіх режимів. Усі елементи задовольняють предикат, тому `all_of` проходить весь масив.

Предикати: легкий $x > 0$; важкий $\sin^2 x + \cos^2 x > 0.5$. Час у мс, середнє з 3 запусків, з прогрівом.

Системні характеристики: процесор (кількість потоків) - 4, режим компіляції: `Release x64`, стандарт мови: `C++20`

1) Бібліотечний `all_of(Release/O2)`

Легкий предикат

[Standard all_of, time in ms]					
N	no policy	seq	par	unseq	par_unseq
100000	0.060	0.088	0.059	0.099	0.097
500000	0.336	0.575	0.227	0.601	0.168
1000000	0.643	0.884	0.280	0.885	0.304
5000000	3.293	4.500	1.473	4.704	1.356

Важкий предикат

[Standard all_of, time in ms]					
N	no policy	seq	par	unseq	par_unseq
100000	4.256	4.311	1.944	4.265	1.894
500000	21.331	21.202	8.401	21.404	8.870
1000000	42.552	42.886	17.273	43.515	17.126
5000000	212.732	213.281	88.956	218.208	89.359

2) Власна реалізація (вибрані K)

[Custom all_of, time in ms depending on K; hardware concurrency = 4]					[Custom all_of, time in ms depending on K; hardware concurrency = 4]				
K	100000	500000	1000000	5000000	K	100000	500000	1000000	5000000
1	0.266	0.543	0.911	3.857	1	4.576	21.747	43.094	213.121
2	0.265	0.377	0.616	2.759	2	2.902	12.432	23.695	123.596
3	0.270	0.395	0.642	2.359	3	2.360	10.560	20.435	96.619
4	0.481	1.108	0.800	2.765	4	2.346	8.676	17.685	97.478
5	0.454	0.696	0.765	2.399	5	3.034	13.217	26.451	111.521
6	0.487	0.555	1.207	2.305	6	2.164	10.080	20.406	87.062
7	0.591	0.575	0.810	2.647	7	2.162	9.956	26.066	120.051
8	0.433	0.548	0.659	2.510	8	1.957	9.113	17.344	85.666
12	0.666	0.746	0.989	2.110	12	2.411	9.860	17.519	84.512
16	1.117	0.843	0.986	2.343	16	2.337	9.772	17.509	84.726
32	1.951	1.570	1.781	2.860	32	2.793	9.522	18.250	84.494

легкий предикат

важкий предикат

Найкраще K (повна таблиця є в `out_O2.txt`):

предикат	$N = 10^5$	$N = 5 \cdot 10^5$	$N = 10^6$	$N = 5 \cdot 10^6$
легкий	2	2	2	12
важкий	8	4	8	32

K/hw(легк.)	0.5	0.5	0.5	3.0
K/hw(важк.)	2.0	1.0	2.0	8.0

Вплив оптимізації

предикат	алгоритм	O0	O2	O0/O2
легкий	no policy	18.649	3.293	5.7×
легкий	par	10.899	1.473	7.4×
легкий	власний(найкраще K)	8.993	2.110	4.3×
важкий	no policy	799.058	212.732	3.8×
важкий	par	224.208	88.956	2.5×
важкий	власний(найкраще K)	227.333	84.494	2.7×

Найкраще K при O0 лежить близько до кількості апаратних потоків: для легкого предиката K = 5, 7, 4, 4 (K/hw = 1–1.75), для важкого K = 8, 8, 8, 16 (K/hw = 2–4). При O2 для легкого предиката найкраще K було меншим, приблизно 0.5·hw. Без оптимізації всі алгоритми працюють у 2.5-7.4 раза повільніше, найбільше для легкого предиката (par: у 7.4 раза). Корисної роботи на кожен елемент стає більше, а витрати на запуск потоків лишаються тими самими. Тому при O0 власна реалізація виграє навіть на легкому предикаті: 8.993 мс проти 18.649 мс послідовно ($\approx 2.1\times$), а при O2 виграш лише $\approx 1.6\times$.

Висновки: політики seq і unseq не прискорюють роботу, час майже такий самий, як без політики. Політики par і par_unseq виграють на великих масивах: приблизно в 2.4 раза (важкий предикат: 212.7 → 89.0 мс, легкий: 3.293 → 1.473 мс при $N = 5 \cdot 10^6$). На малому масиві з легким предикатом виграшу немає, бо запуск потоків коштує стільки ж, скільки сама робота.

У власній реалізації найкраще K для важкого предиката дорівнює кількості апаратних потоків або кратне їй (K/hw = 1–2), а далі час майже не змінюється (≈ 85 мс). Для легкого предиката на малих масивах найкраще K = 2 (K/hw = 0.5). Поки K менше за hw, час спадає приблизно як 1/K. Коли K більше за hw, для важкого предиката час виходить на плато, а для легкого зростає лінійно, приблизно на 0.05-0.06 мс на кожен додатковий потік. Власна реалізація для важкого предиката не гірша за par (84.5 проти 89.0 мс), а для легкого програє (2.11 проти 1.47 мс), бо потоки створюються щоразу, а par використовує готовий пул. Без оптимізації (O0) усе працює в 2.5-7.4 раза повільніше, ніж з O2.